

Informe de análisis y diseño — SGR

Documento generado desde `informe.md` del repositorio

Sistema de Gestión de Resultados para las delegaciones municipales de La Serena

Asignatura: Proyecto Integrador · INACAP **Equipo:** Origami SpA — Tomás Ibacache · Héctor (completar apellido antes de entregar) **Entrega:** primera evaluación de Análisis y Diseño — 15 de septiembre de 2026
Repositorio: <<https://github.com/TIbacache/matriz-sgr>> (privado; acceso concedido al docente) **Rama de la entrega:** entrega/analisis-diseno

Índice

1. Resumen ejecutivo
2. El caso y su alcance
3. De dónde sale cada cosa: la jerarquía de fuentes
4. Los artefactos, criterio por criterio
5. Trazabilidad
6. 5.1 RF → caso de uso
7. 5.2 Caso de uso → pantalla del mockup
8. 5.3 Caso de uso → clase y servicio
9. 5.4 Caso de uso → tabla del DER
10. Cómo se comprueba la coherencia
11. Decisiones de diseño que hay que poder defender
12. Lo que el sistema declara y no arregla
13. Tecnologías
14. Evidencias y cómo revisarlas
15. Consultas abiertas al docente

1. Resumen ejecutivo

SGR reemplaza la planilla «Matriz SGR» con la que hoy las delegaciones municipales de La Serena miden el trabajo de sus funcionarios. Este informe reúne los artefactos de análisis y diseño de la primera evaluación y, sobre todo, **la trazabilidad entre ellos**, que es el criterio transversal de la rúbrica:

«Los artefactos no deben presentarse como piezas aisladas. Debe existir trazabilidad entre requerimientos, casos de uso, clases, entidades de base de datos y pantallas del mockup, cuando corresponda.»

Lo que se entrega, en números:

Requerimientos	38 funcionales y 18 no funcionales , con código, actor, módulo y épica
Casos de uso	12 , cada uno con diagrama y ficha de doce campos (la rúbrica exige 10 como mínimo)
Relaciones UML	3 «include» y 6 «extend»
Clases	22 del dominio y 12 servicios , en 5 diagramas
Modelo de datos	22 tablas, 52 claves foráneas , 8 enumerados, 11 UNIQUE y 5 CHECK
Script SQL	Ejecutable, probado en MySQL 8.0 y MariaDB 10.4 y 11.4
Mockup	17 pantallas : 8 del camino feliz y 9 escenarios alternativos
Diagramas	32 , todos en PlantUML con su fuente .pum1 versionada
Comprobaciones automáticas	311 de coherencia entre artefactos, más 332 del software funcionando

El mockup no es un dibujo: es la aplicación real, congelada. El sistema está construido y corriendo — API, base de datos, tiempo real y siete pantallas—, y cada imagen de este informe salió de operarlo. La rúbrica §3 no exige integración con base de datos ni consumo de API para esta entrega; que el prototipo la tenga no da puntos extra, pero permite afirmar cada cosa que este documento afirma.

2. El caso y su alcance

Las delegaciones municipales de La Serena atienden vecinos y registran su trabajo en una planilla compartida de Google Sheets. El problema no es la planilla: es lo que la planilla **no puede hacer**.

El cliente lo resumió con un caso concreto: **un niño pidió el mismo regalo de Navidad en cinco delegaciones distintas y el sistema no lo detectaba**. Cada delegación llevaba su propio libro, la persona atendida no era una entidad del modelo, y nadie podía cruzar la información sin abrir cinco archivos.

De ahí salen las tres capacidades que ordenan el diseño:

1. **Medir el desempeño de forma comparable** — metas y ponderadores por funcionario, ítem y período, con un solo motor de cálculo y un semáforo con umbrales configurables.
2. **Sostener la medición con evidencia** — una actividad solo suma cuando su evidencia está **validada por otra persona**, con código único e inmutable.
3. **Seguir a la persona atendida entre delegaciones** — el RUT es único por organización y no por delegación, así que la duplicidad es una propiedad del modelo y no un informe posterior.

Alcance de esta iteración

23 de los 38 RF están implementados y verificados; 8 están parciales y 7 quedan fuera por decisión de alcance (§8). Los doce casos de uso salen de **lo construido**, no del catálogo completo de requerimientos: si se documentaran casos de funciones que no existen, el mapa caso de uso → pantalla quedaría con huecos y el criterio transversal de coherencia se caería por el peor motivo posible.

3. De dónde sale cada cosa: la jerarquía de fuentes

Cuando dos fuentes se contradicen, gana la de más arriba, y la decisión se registra:

	Fuente	Qué aporta
1	PDF oficial del proyecto (profesores)	Los 38 RF, 18 RNF, 13 RN, 10 CA y 31 historias. Es la especificación que se evalúa. Transcrito en requerimientos-oficiales.md
2	Estructura de la planilla real	Columnas, catálogos, códigos y fórmulas de la herramienta en producción (estructura-planilla-real.md)
3	Reunión con el cliente	Citas con marca de tiempo (anotaciones-clase.md)
4	Visión inicial del equipo	Anterior al PDF; se conserva con su nota de vigencia

El PDF lo dice explícitamente, y es la regla que gobierna este informe:

«Si una historia contradice un requerimiento formal, prevalece el requerimiento y debe registrarse la decisión. **Las ambigüedades no se resuelven en silencio**: se documentan como supuestos o consultas al docente.»

Por eso hay **catorce consultas abiertas** (§11) y **cuatro desvíos declarados** (§8). Ninguno se disimuló.

4. Los artefactos, criterio por criterio

#	Criterio	Pts	Artefacto	Diagramas
1	Planner	15	guia-planner-hector.pdf · planner-delta.md · plan-desarrollo.csv	—
2	Diagramas de requerimientos	10	requerimientos.md	01 a 08
3	Caso de uso general	10	casos-uso-general.md	09 , 10
4	Casos de uso específicos y fichas	20	casos-uso-detalle.md	11 a 22
5	Diagrama de clases	15	clases.md	23 a 27
6	DER MySQL	10	der.md	28 a 32
7	Script SQL	10	script-sql.md · sgr-mysql.sql	—
8	Mockup funcional y Git	10	docs/mockups/ con su README	17 pantallas

Los 32 diagramas están en PlantUML, con la fuente [.puml](#) versionada en [puml/](#) y el PNG generado desde ella. Ningún PNG se edita a mano: el diagrama y su fuente no pueden divergir. La convención —idioma, paleta, tipografía— está en [puml/estilo.md](#).

Todo va **en español**, salvo los estereotipos UML/SysML («requirement» , «include» , «extend» , «satisfy» ...), que conservan su nombre estándar porque son **notación, no texto**. Por la misma razón no

se traducen la visibilidad (+ - #), las cardinalidades ni los tipos de dato.

5. Trazabilidad

Esta sección es el corazón del informe. Son cuatro tablas y **se recorren en las dos direcciones**: desde un requerimiento hasta la tabla que lo sostiene, y desde una tabla de vuelta hasta el requerimiento que la justifica.

Requerimiento → Caso de uso → Pantalla del mockup
↓
Clase + servicio → Tabla del DER → Script SQL

5.1 RF → caso de uso

Los 38 requerimientos funcionales con el caso de uso que los cumple. La tabla completa, con actor, módulo y épica, está en [requerimientos.md §12](#) ; aquí va el cruce.

Legenda de estado —  implementado y verificado ·  parcial, con su motivo en §8 ·  fuera del alcance de esta iteración. En negrita, el caso de uso principal del requerimiento; los **CU-I** y **CU-E** son los casos incluidos y de extensión.

RF	Requerimiento	Caso de uso	Estado
RF-001	Administrar delegaciones	—	
RF-002	Administrar usuarios y roles	—	
RF-003	Configurar cargos y sus ítems	—	
RF-004	Catálogo de actividades y atenciones	CU-06, CU-08	
RF-005	Configurar períodos	—	
RF-006	Configurar ponderaciones	CU-04	
RF-007	Configurar metas y umbrales versionados	CU-04	
RF-008	Ficha personal	CU-05	
RF-009	Registrar actividades	CU-01	
RF-010	Validar obligatoriedad, formato y coherencia	CU-01 , CU-06, CU-I1	
RF-011	Código de evidencia único e inmutable	CU-01 , CU-I2	
RF-012	Asociar la evidencia al código	CU-02	
RF-013	Validar la evidencia	CU-03 , CU-E2	
RF-014	Solo lo validado suma	CU-03	
RF-015	Atención social con hasta tres gestiones	CU-08	
RF-016	Crear compromisos internos y externos	CU-06	
RF-017	Asignar solicitante, territorio y área	CU-06	

RF	Requerimiento	Caso de uso	Estado
RF-018	Cuatro estados controlados	CU-07	● D-b
RF-019	Control de plazos	CU-07	● D-b
RF-020	El cierre alimenta el indicador una vez	—	■
RF-021	Resumen colectivo por funcionario y estado	—	■
RF-022	Calcular avance por ítem, funcionario y período	CU-09	✓
RF-023	Porcentaje de cumplimiento	CU-05	✓
RF-024	Cumplimiento ponderado con tope	CU-09	✓
RF-025	Incentivos y penalizaciones	—	■
RF-026	Meta esperada al día	CU-05	✓
RF-027	Semáforo con umbrales configurables	CU-05, CU-E6	✓
RF-028	Tablero personal	—	■
RF-029	Tablero de delegación consolidado	CU-09, CU-E6	✓
RF-030	Actividad reciente y días sin ingreso	CU-12, CU-E5	✓
RF-031	Vista global por cargos	—	■
RF-032	Buscar y filtrar	CU-11, CU-E1	✓
RF-033	Generar y exportar informes	—	■
RF-034	Trabajo simultáneo sin sobrescritura silenciosa	CU-07, CU-E4	●
RF-035	Comentarios y observaciones	—	■
RF-036	Trazabilidad de altas y modificaciones	CU-10, CU-I3	● D-c
RF-037	Alertas de vencimientos y evidencias pendientes	—	●
RF-038	Versionar parámetros	—	●

38 RF — 23 ✓ · 8 ● · 7 ■.

Un RF sin caso de uso no es un hueco. RF-001, RF-003 y RF-005 son configuración administrativa: su API está implementada y verificada, pero **la pantalla de administración no está construida**, así que no se les inventó un caso de uso para rellenar la tabla —un caso de uso sin pantalla dejaría el mapa del §5.2 con un agujero—. RF-002 y RF-038 están parciales por la misma razón. Los siete ■ están declarados fuera de alcance en §8 y aparecen con **borde punteado** en el diagrama de requerimientos: ese diagrama dice qué necesita el sistema, no qué alcanzamos a construir.

5.2 Caso de uso → pantalla del mockup

La exigencia literal del documento «Relación entre los artefactos»:

«Los mockups deben representar las pantallas necesarias para ejecutar los casos de uso del diagrama y también contemplar los escenarios alternativos modelados.»

Son dos mapas porque son dos cosas: los doce casos principales y los nueve escenarios alternativos.

Los doce casos de uso

CU	Caso de uso	Actor principal	Pantalla
CU-01	Registrar actividad diaria	Funcionario	<u>03-ficha</u>
CU-02	Adjuntar evidencia a una actividad	Funcionario	<u>03-ficha</u>
CU-03	Validar o rechazar una evidencia	Verificador	<u>04-verificacion</u>
CU-04	Configurar metas y ponderadores	Administrador · Coordinador	<u>05-metas</u>
CU-05	Consultar la ficha personal y el semáforo	Funcionario	<u>03-ficha</u>
CU-06	Registrar un compromiso del vecino en el tubo	Funcionario · Delegado	<u>02-tubo</u>
CU-07	Mover un compromiso de estado	Funcionario · Delegado	<u>02-tubo</u>
CU-08	Registrar una atención social y sus gestiones	Funcionario	<u>03-ficha</u> , modal del caso
CU-09	Consultar el tablero consolidado	Delegado · Usuario de consulta	<u>07-dashboard</u>
CU-10	Anular una actividad con motivo	Funcionario · Delegado	<u>03-ficha</u>
CU-11	Buscar el historial de un vecino entre delegaciones	Funcionario · Coordinador	<u>06-vecino</u>
CU-12	Controlar la actividad de usuarios	Administrador · Coordinador	<u>08-actividad</u>

La octava pantalla, 01-login , es el ingreso: no tiene caso de uso propio porque es precondition de los doce.

Los nueve escenarios alternativos

ID	Escenario	Tipo	Mockup
CU-I1	Validar los datos del registro	«include»	<u>09-alt-datos-invalidos</u>
CU-I2	Generar el código único de la actividad	«include»	<u>03-ficha</u> — sin pantalla propia: el código no se pide, lo genera el servidor

ID	Escenario	Tipo	Mockup
CU-I3	Registrar en la bitácora de auditoría	«include»	Sin pantalla — RF-036 pendiente (desvío D-c)
CU-E1	Avisar posible atención duplicada	«extend»	<u>10-alt-duplicidad</u>
CU-E2	Exigir observación de la decisión	«extend»	<u>11-alt-observacion-obligatoria</u>
CU-E3	Rechazar la validación propia	«extend»	<u>12-alt-validacion-propia</u>
CU-E4	Informar conflicto de versión	«extend»	<u>13-alt-conflicto-version</u>
CU-E5	Denegar por alcance, con el motivo escrito	«extend»	<u>14-alt-alcance-vecino</u> y <u>15-alt-alcance-actividad</u>
CU-E6	Informar delegación sin medición	«extend»	<u>16-alt-sin-medicion</u>

El mapa con el detalle de qué se ve en cada uno está en casos-uso-general.md §8.1. Se agrega una décima captura, 17-alt-alcance-reducido, que no es uno de los nueve pero es el mismo tipo de escenario: para un delegado, la atención de otra delegación viaja con su fecha, su delegación y su tipo, sin el detalle.

Dos casos incluidos no tienen imagen, y las dos ausencias son decisiones. CU-I2 porque el código único no es una pantalla sino un dato que ya se ve asignado en la ficha. **CU-I3 porque la pantalla no existe:** RF-036 pide trazabilidad consultable, el sistema audita toda escritura crítica pero **falta la pantalla para leer la bitácora.** Dibujarle un mockup habría hecho desaparecer del artefacto justo lo que falta construir.

Ninguna de las 17 imágenes está simulada. El generador abre la pantalla con la cuenta que corresponde y **opera la aplicación:** escribe en el campo, sale de él, aprieta el botón. Donde el escenario necesita una segunda persona —el conflicto de versión— abre una segunda sesión contra la API que guarda primero. Si en una imagen se lee un 403 o un 409, es porque el servidor lo devolvió.

5.3 Caso de uso → clase y servicio

Las clases del dominio son persistentes y deliberadamente delgadas: **el comportamiento vive en los servicios**, porque la lógica es transversal a varias entidades —el cumplimiento cruza meta, actividad, evidencia, validación y ausencia— y meterla dentro de una de ellas la escondería.

Las tres clases de identidad — `Organization`, `User` y `OrganizationMember` — participan en **los doce** casos: toda consulta filtra por organización y todo alcance se resuelve por rol y delegación. No se repiten en cada fila.

La columna de servicios sale del endpoint que implementa cada caso, no del archivo que lo contiene. Un módulo de rutas suele atender varios casos de uso — `actividades.routes.ts` sostiene CU-01, CU-02, CU-08 y CU-10—, así que atribuirle a todos los servicios que el archivo importa sobreestimaría la dependencia. Por eso CU-01 no lista `ServicioConcurrencia`: el alta crea, no actualiza, y el bloqueo optimista aparece recién al modificar (CU-07, CU-10).

CU	Clases del dominio	Servicios
CU-01	Periodo , Cargo , ItemMedicion , Actividad , PersonaUsuaría , Tarea , Auditoria	ServicioCodigos , ServicioAuditoria , ServicioBroadcast
CU-02	Actividad , Evidencia , CatalogoItem , Parametro , Auditoria	ServicioAlmacenamiento , ServicioAlcance , ServicioAuditoria , ServicioBroadcast
CU-03	Evidencia , Validacion , Auditoria	ServicioAlcance , ServicioAuditoria , ServicioBroadcast
CU-04	Periodo , Cargo , ItemMedicion , MetaItem , Auditoria	ServicioConcurrencia , ServicioAlcance , ServicioAuditoria , ServicioBroadcast
CU-05	Periodo , Parametro , ItemMedicion , MetaItem , Validacion , Ausencia	ServicioCumplimiento , ServicioParametros
CU-06	UnidadTerritorial , CategoriaGestion , Tarea , PersonaUsuaría , CatalogoItem , Auditoria	ServicioVecinos , ServicioAlcance , ServicioAuditoria , ServicioBroadcast
CU-07	UnidadTerritorial , Tarea , Auditoria	ServicioConcurrencia , ServicioAlcance , ServicioAuditoria , ServicioBroadcast
CU-08	Actividad , AtencionSocial , CatalogoItem , Auditoria	ServicioAtencionSocial , ServicioAlcance , ServicioAuditoria , ServicioBroadcast , ServicioConcurrencia
CU-09	UnidadTerritorial , Periodo , Parametro , MetaItem , Validacion , Ausencia	ServicioCumplimiento , ServicioParametros
CU-10	Actividad , Auditoria	ServicioConcurrencia , ServicioAuditoria , ServicioBroadcast
CU-11	PersonaUsuaría , Actividad , AtencionSocial , Parametro , Auditoria	ServicioVecinos , ServicioAlcance , ServicioParametros , ServicioAuditoria , ServicioConcurrencia , ServicioBroadcast
CU-12	UnidadTerritorial , Periodo , Parametro , Auditoria	ServicioActividadUsuarios , ServicioPresencia , ServicioParametros , ServicioAuditoria

ServicioParametros aparece por todas partes, y es una regla del proyecto hecha dibujo: ningún valor de negocio vive en el código. El tope de cumplimiento, los umbrales del semáforo, la ventana de duplicidad y el tamaño máximo de evidencia salen de la tabla `parametros`, con vigencia por período (RNF-015, RF-038).

ServicioAuditoria aparece en diez de los doce, incluidos CU-11 y CU-12, que solo consultan: abrir la ficha de un vecino deja registro porque es un dato personal identificado (Leyes 19.628 y 21.719, y Ley 21.663 de ciberseguridad, que exige trazabilidad del **acceso** y no solo de la modificación). Los dos que no auditan son **CU-05 y CU-09**, y por la misma razón: son lecturas de datos **agregados o propios** —el funcionario mirando su ficha, el tablero consolidado— donde no hay un dato personal de tercero que justifique el registro. Auditar toda lectura de un tablero llenaría la bitácora de ruido y haría más difícil encontrar el acceso que sí importa.

Tres clases del modelo **no tienen caso de uso**, y no es un olvido: `Ajuste` (RF-025) y `Comentario` (RF-035) están fuera de alcance, y `TareaHistorial` tiene tabla pero la aplicación nunca escribe en ella (§8, desvío D-b). Las tres van con **borde punteado** en el diagrama de clases.

5.4 Caso de uso → tabla del DER

22 clases, 22 tablas: no hay clase sin tabla ni tabla sin clase, y la correspondencia se comprueba automáticamente en los dos sentidos contra el esquema real.

CU	Tablas del DER
CU-01	<code>periodos</code> , <code>cargos</code> , <code>items_medicion</code> , <code>actividades</code> , <code>personas_usuarias</code> , <code>tareas</code> , <code>auditoria</code>
CU-02	<code>actividades</code> , <code>evidencias</code> , <code>catalogo_items</code> , <code>parametros</code> , <code>auditoria</code>
CU-03	<code>evidencias</code> , <code>validaciones</code> , <code>auditoria</code>
CU-04	<code>periodos</code> , <code>cargos</code> , <code>items_medicion</code> , <code>metas_item</code> , <code>auditoria</code>
CU-05	<code>periodos</code> , <code>parametros</code> , <code>items_medicion</code> , <code>metas_item</code> , <code>validaciones</code> , <code>ausencias</code>
CU-06	<code>unidades_territoriales</code> , <code>categorias_gestion</code> , <code>tareas</code> , <code>personas_usuarias</code> , <code>catalogo_items</code> , <code>auditoria</code>
CU-07	<code>unidades_territoriales</code> , <code>tareas</code> , <code>auditoria</code>
CU-08	<code>actividades</code> , <code>atenciones_sociales</code> , <code>catalogo_items</code> , <code>auditoria</code>
CU-09	<code>unidades_territoriales</code> , <code>periodos</code> , <code>parametros</code> , <code>metas_item</code> , <code>validaciones</code> , <code>ausencias</code>
CU-10	<code>actividades</code> , <code>auditoria</code>
CU-11	<code>personas_usuarias</code> , <code>actividades</code> , <code>atenciones_sociales</code> , <code>parametros</code> , <code>auditoria</code>
CU-12	<code>unidades_territoriales</code> , <code>periodos</code> , <code>parametros</code> , <code>auditoria</code>

A las que se suman `organizations`, `users` y `organization_members` en los doce, y las tres sin caso de uso: `ajustes`, `comentarios` y `tarea_historial`.

Toda tabla de esta columna existe en el script SQL con sus claves foráneas, que es la validación de consistencia que la rúbrica exige de forma explícita:

«Toda FK representada en el DER debe existir en el script SQL. Del mismo modo, las tablas creadas en el script deben corresponder al modelo presentado en el informe.»

Las **52 claves foráneas** no se transcribieron: se extrajeron de las migraciones una por una, con su `ON DELETE` (37 `CASCADE`, 7 `RESTRICT`, 8 `SET NULL`), y el verificador las compara contra el script. El detalle está en [der.md §9](#).

6. Cómo se comprueba la coherencia

Revisar a ojo la trazabilidad cada vez que cambia un artefacto no escala, y el criterio transversal castiga la incoherencia **en varios criterios a la vez**. Por eso la coherencia se comprueba con un script:

```
cd frontend
npm run verificar:entrega
```

311 comprobaciones, sin tocar la red. Entre otras:

- La trazabilidad RF ↔ CU **en los dos sentidos**: lo que dice la tabla del criterio 2 y lo que dice la del criterio 3 tienen que coincidir.
- Que cada caso de uso tenga su pantalla y que esa pantalla exista como archivo.
- Que cada «include» y cada «extend» tenga su casilla en el mapa, que la casilla apunte a una imagen que existe **y que el generador de mockups efectivamente la produzca**.
- El diagrama de clases contra `schema.prisma` en los dos sentidos: ninguna clase inventada, ningún modelo sin dibujar.
- **El DER contra las migraciones**: las 52 claves foráneas, una por una, con su `ON DELETE`.
- **El script SQL contra el esquema**: las 22 tablas, sus columnas una por una en los dos sentidos, las 52 FK, el orden de creación, los 3 disparadores y los 5 `CHECK`.
- La convención de los 32 diagramas: tipografía Arial, ningún rojo institucional en zona de datos, PNG generado y entrada en el índice.

Sale con código 1 si algo se cae, así que sirve como control antes de cada entrega.

A esas 311 se suman **332 comprobaciones del software funcionando**: 21 de integración, 37 del motor de cálculo y 191 de la API y su alcance por rol —249 en el backend—, más 83 de contraste WCAG en el frontend, y aparte las de normalización de RUT. Son cosas distintas y no se suman entre sí: las 332 comprueban que **el sistema funciona**; las 311, que **los artefactos dicen lo mismo entre sí**.

7. Decisiones de diseño que hay que poder defender

7.1 El DER y el script van en MySQL; el sistema corre en PostgreSQL 16

La rúbrica pide el modelo en MySQL. El sistema está construido sobre **PostgreSQL 16**, y migrarlo por un criterio de formato sería cambiar la arquitectura de un producto que funciona.

Decisión: el DER y el script se entregan en MySQL, **traducidos fielmente desde el esquema real**, declarando las equivalencias de tipos ([der.md §12](#)). El script **se ejecutó de verdad** en MySQL 8.0.46 y en MariaDB 10.4.34 y 11.4.13 —MariaDB porque es lo que trae XAMPP, que es lo más probable que tenga a

mano quien revise—, con cero errores en los tres y **trece sentencias que deben fallar fallando con el error correcto**. Es la consulta abierta nº 13.

Una traducción que hay que poder defender: `timestampz` → `DATETIME(3)` **pierde la zona horaria**. El equivalente fiel sería `TIMESTAMP`, pero su rango termina en 2038. Se eligió `DATETIME(3)` y se declara que la aplicación guarda **todo instante en UTC**.

7.2 No se dibuja generalización ni herencia

La rúbrica pide `«include»`, `«extend»` o generalización entre actores, y herencia entre clases **solo con justificación en el diseño**. En SGR no la hay, y afirmarla sería exactamente la incoherencia que el criterio transversal castiga:

- **Los seis roles no forman una jerarquía**: son valores de un enumerado que se **solapan**. El Verificador valida evidencias pero no ve el tubo; el Coordinador y el Administrador comparten el panel de actividad sin que uno herede del otro.
- **Las entidades no comparten ancestro**: todas llevan `organizationId` y casi todas `version`, pero eso es una convención transversal, no una superclase.

Es **una sola decisión** tomada en dos artefactos: cambiarla obliga a cambiar los dos. Lo que sí se usa de verdad son **tres** `«include»` y **seis** `«extend»`.

7.3 El DER usa notación crow's foot, no cajas de clase

El DER y el diagrama de clases se evalúan por separado. Entregar dos veces el mismo dibujo con otro título es la forma más barata de perder los dos, así que el DER usa `entity` y `||--o{`, con sus claves y cardinalidades, y el de clases usa cajas con visibilidad `+ - #`, atributos, operaciones y multiplicidades.

7.4 Un solo cálculo, sin segunda verdad

El cumplimiento se mide **por funcionario** en un único servicio, y la delegación es el **promedio simple** de su gente. Hubo una vista materializada en SQL que calculaba lo mismo por otro camino: **se eliminó**, junto con su tabla y el cron que la refrescaba. Dos fuentes para un mismo número son dos números que tarde o temprano difieren.

De ahí sale una regla de presentación que parece un detalle y no lo es: **una delegación sin nadie con metas configuradas no cumple 0%, no tiene medición**. Un 0% diría que trabajaron y no cumplieron; la verdad es que no hay nada que medir. Es el escenario CU-E6.

7.5 Los datos de demostración son parte del entregable

Los datos son **100% ficticios** —lo exige el PDF y también la ley— y están armados a propósito para que las reglas se puedan **mostrar**, no solo afirmar. Hay tres casos deliberados:

- Una delegación **sin medición**, que es lo que hace capturable CU-E6.
- Una funcionaria **con metas y cero actividades**, que es el «sin registro» del panel de control.
- Una evidencia **subida por el coordinador**, que por eso mismo no puede validarla: es CU-E3, y sin ese caso la segregación de funciones quedaba implementada pero imposible de demostrar en pantalla.

8. Lo que el sistema declara y no arregla

Un artefacto que describe el código como si fuera el requisito **vuelve invisible el incumplimiento**. Estos cuatro desvíos están declarados en las fichas de los casos de uso, en el DER y en este informe.

	Qué pide el requerimiento	Qué hay hoy
D-a	RF-016 asigna la creación de compromisos al Funcionario y al Delegado	El alta está restringida a jefatura y nivel central. El funcionario mueve sus compromisos, pero no los crea. Viene de una matriz de permisos de la fuente más baja de la jerarquía, que quedó por encima del RF sin que nadie lo decidiera
D-b	RF-018 pide cuatro estados con historial de transiciones y RF-019 alertas de «próximo a vencer»	Hay tres estados y solo se marcan los vencidos. La tabla <code>tarea_historial</code> existe y los datos de demostración la llenan, pero la aplicación nunca escribe en ella : en la demostración el historial se ve poblado y en uso real no se llenaría
D-c	RF-036 pide trazabilidad consultable	Se audita toda escritura crítica —usuario, fecha, acción, entidad, valor anterior y nuevo— pero falta la pantalla para leer la bitácora . Es la razón por la que CU-I3 no tiene mockup
D-d	Integridad referencial	<code>periodos.cerrado_por_id</code> debería ser clave foránea a <code>users</code> y no lo es . No estaba declarado en ninguna parte: apareció al extraer las 52 FK de las migraciones para el DER. Ni el DER ni el script la agregan, porque el artefacto describe el sistema que hay. El caso análogo, <code>ajustes.registrado_por_id</code> , sí la tiene con <code>RESTRICT</code>

Los siete RF fuera de alcance

No son olvidos: son una decisión para llegar con lo esencial construido y verificado. Ninguno está en el alcance mínimo exigido del PDF.

RF	Por qué queda fuera
RF-020, RF-021	Dependen de RF-018, que necesita el cuarto estado y el historial de transiciones. Es un cambio de modelo, no una pantalla
RF-025	La entidad existe en el modelo, sin API. Afecta al cálculo, y el cálculo debía estabilizarse primero
RF-028	La ficha personal ya entrega lo esencial; el tablero personal es presentación sobre datos que ya se calculan
RF-031	El consolidado ya agrupa por área del cargo; la vista por cargos es otro corte del mismo motor
RF-033	Exportación. No aporta a la evaluación de análisis y diseño y sí consume tiempo de formato
RF-035	La entidad existe en el modelo, sin API

9. Tecnologías

Capa	Elección	Por qué
Backend	Node.js · TypeScript estricto · Express 5	Un solo lenguaje en las dos puntas, con tipos comprobados en compilación
Base de datos	PostgreSQL 16 · Prisma ORM	Restricciones, disparadores y <code>CHECK</code> en la base, no solo en el código

Capa	Elección	Por qué
Tiempo real	Socket.io, con salas por delegación y por organización	El tubo de trabajo es colectivo: dos personas moviendo la misma tarjeta tienen que verse
Frontend	React · TypeScript · Vite	—
Estilos	CSS3 plano con tokens propios , sin framework de UI	La identidad gráfica municipal es normativa: el rojo institucional, la tipografía y el contraste no son preferencias. Un framework de terceros habría que pelearlo en vez de usarlo
Interacción	dnd-kit (arrastre del kanban) · ECharts (gráficos)	Las dos librerías externas del proyecto, ambas donde aportan
Autorización	JWT por rol, multi-organización	Toda consulta filtra por organización; un recurso de otra responde 404, no 403

Costo cero: no hay ninguna dependencia ni servicio de pago en el proyecto.

Accesibilidad y contraste se comprueban por script (83 comprobaciones WCAG sobre los dos temas, claro y oscuro), y toda la aplicación es operable por teclado. La norma gráfica municipal usa Libre Franklin y General Sans en pantalla y **Arial en lo impreso**, que es la tipografía de este informe y de los 32 diagramas.

10. Evidencias y cómo revisarlas

Repositorio

<<https://github.com/Tlbacache/matriz-sgr>> — rama `entrega/analisis-diseno`.

Contiene el mockup funcional, **los archivos editables de todos los diagramas** (`docs/entrega/puml/*.puml`) y el código completo del sistema. La estructura de la entrega está en `docs/entrega/README.md`.

El mockup

Cada pantalla está **dos veces**, y cada formato sirve para algo distinto:

Formato	Para qué
<code>.html</code>	Se adjunta a la tarea de Planner. Se abre con doble clic —sin instalar nada, sin servidor, sin base de datos y sin internet — y se ve exactamente como la aplicación
<code>.png</code>	Para que la pantalla se vea en GitHub , donde Markdown muestra imágenes pero no ejecuta HTML

Los `.html` son **estáticos**: los botones no responden, y cada archivo lo dice en una franja al pie para que nadie crea que algo está roto. Que sean autocontenidos está comprobado: un script los abre desde `file://` **con la red bloqueada** y verifica que llevan su CSS dentro, que no queda ningún `<script>`, que todas las imágenes cargan y que la identidad visual se conserva sin conexión.

El índice con las 17 pantallas comentadas una por una está en `docs/mockups/README.md`.

Planner

⚠ **Pendiente al momento de redactar este informe.** Aquí va la captura del tablero con las 87 tareas cargadas, y su enlace.

La carga es **manual y está documentada paso a paso** en [guia-planner-hector.pdf](#), en seis tandas. No se automatiza, y no por comodidad: INACAP tiene desactivado el consentimiento de usuario para *Microsoft Graph Command Line Tools*, así que ningún script puede crear las tareas. Se intentó con el permiso mínimo y con código de dispositivo, y siempre termina en «Need admin approval»; el diagnóstico está en [guia-cargar-planner.md](#).

Sobre los estados: Planner básico ofrece solo *No iniciada / En curso / Completada*, y la rúbrica menciona cuatro. El cuarto —«En revisión»— se cubre con **etiqueta de color**, que es el mecanismo que la herramienta permite.

11. Consultas abiertas al docente

El PDF exige que las ambigüedades se documenten en vez de resolverse en silencio. Hay **catorce**, con el mismo formato cada una: qué dice cada fuente, qué se hizo mientras tanto y qué cambiaría con la respuesta. Están en [requerimientos-oficiales.md §10](#). Las tres que más afectan a esta entrega:

nº	Consulta	Qué se hizo mientras tanto
12	La ventana de duplicidad entre atenciones de la misma persona en distintas delegaciones no tiene valor definido	El parámetro existe y es configurable; la pantalla declara el valor como provisional en vez de presentarlo como definitivo
13	El modelo se pide en MySQL y el sistema corre en PostgreSQL	Se entrega el modelo traducido, con las equivalencias declaradas, y no se migra el sistema (§7.1)
8	Una validación aprobada, ¿se puede revertir?	No se revierte: una aprobación ya sumó al puntaje. Para corregir, se anula la actividad con motivo , que deja registro en la bitácora

Informe de análisis y diseño — SGR · Equipo Origami SpA · 10 de septiembre de 2026